

Sajan

Date:

Page:

Bayesian Networks:

Prob. axioms: $P(A) \geq 0$

$$P(\text{Universal Set}) = 1$$

for disjoint events, $P(\cup A_i) = \sum_i P(A_i)$
 $\rightarrow P(E \cap F) = 0$

Inclusion-
exclusion
principle

$$P(E \cup F) = P(E) + P(F) - P(E \cap F)$$

$$P(E \cup F \cup G) = P(E) + P(F) + P(G)$$

$$- P(E \cap F) - P(E \cap G) - P(F \cap G) + P(E \cap F \cap G)$$

Random variable associates a number with an outcome of a random experiment.

Conditional - Joint - Marginal Probabilities
 $[P(A|B)]$ $[P(A, B)]$ $[P(A)]$

$$p(x) = \sum_y p(x, y)$$

$$P(A, B) = P(A|B) P(B)$$

Bayes' theorem:

$$p(y|x) = \frac{p(x|y) P(y)}{P(x)}$$

$X \perp Y$: X, Y are independent;

$$\rightarrow P(x, y) = P(x) P(y)$$

$$\rightarrow P(y|x) = P(y)$$

Conditional Independence:

$$p(x, y|z) = p(x|z) \cdot p(y|z)$$

★ Chain rule:

$$P(x_1, x_2, \dots, x_n) = P(x_1) \cdot \prod_{i=2}^n P(x_i | x_1^{i-1})$$

e.g. toss 3 times

X : no. of heads on 1st toss,

Y : no. of heads on last 2 tosses

F : no. of heads on first 2 tosses.

$Y \backslash X$	0	1
0	1/8	1/8
1	1/4	1/4
2	1/8	1/8

$F \backslash X$	0	1
0	1/4	0
1	1/4	1/4
2	0	1/4

$$P(b) = P(a, b) + P(\neg a, b)$$

→ for n variable size of Joint Prob. Dist. Table is $O(d^n)$, where d is the domain of the variables.

$$P(a|b) + P(\neg a|b) = 1$$

$$\sum_{i=1}^N P(A=a_i | B=b_2) = 1$$

e.g. $P(C, D, A, B) = P(C) P(D|C) P(A|C, D) P(B|C, D, A)$

→ To specify joint distribution, we need $2^n - 1$ values. (given n binary variables).

★ Using conditional parameterization and independence assumptions, we can reduce no. of parameters.

e.g. I, G, S in joint $2 \times 3 \times 2 - 1 = 11$ parameters
 $(2) \quad (3) \quad (2)$

Assume: $(S \perp G) | I$

$$P(I, G, S) = P(I) P(G|I) P(S|G, I) \\ = P(I) P(G|I) P(S|I)$$

no. of parameters = $1 + 4 + 2 = 7$

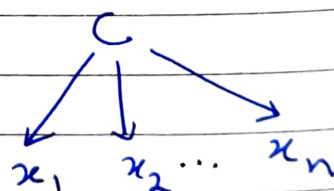
$$i = 0, 1 \\ P(i) = a, b$$

$$g = A, B, C \\ P(g|i) = x_0, y_0, z_0 \\ P(g|i) = x_1, y_1, z_1$$

$$b = 0, 1 \\ P(b|i_0) = p_0, q_0 \\ P(b|i_1) = p_1, q_1$$

Let's use graphs:

Class Conditional Independence (C.C.I) { Assume $x_1, x_2, \dots, x_n$ are independent given C .



Naive Bayes { $P(C, x_1, x_2, \dots, x_n) = P(C) P(x_1|C) P(x_2|C) \dots P(x_n|C)$
 bcz $(x_i \perp x_j) | C$

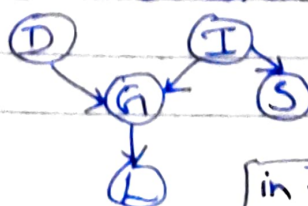
→ Bayesian networks relax this strong assumption.

Graphs: nodes indicate random variables.

edges indicate conditional dependence.

↓ Dependency

e.g.



in full joint table:
 $n = 2 \times 3 \times 2 \times 2 - 1$

in Bay. net.
 $n = 15$

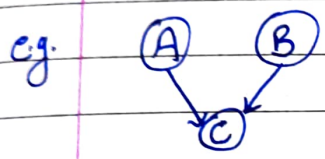
$P(Y, x_1, x_2, \dots, x_d)$, Y can take K values
 x_i can take t values.

For full joint prob. dist:
 no. of parameters = $Kt^d - 1$

With CCI assumption:

$$P = P(Y) P(x_1 | Y) P(x_2 | Y) \dots P(x_d | Y)$$

no. of parameters = $(K-1) + dK(t-1)$



$$P(C=T) = \sum_{A,B} P(A, B, C=T)$$

~~P(A,B,C)~~

$$= \sum_{A,B} P(A) P(B) P(C=T | A, B)$$

$$P(A=T, B=T) = P(A=T) P(B=T)$$

eg. $A \perp B$

$$P(A, B) = P(A) P(B)$$

$$P(A=0, B=0) = 0.1$$

$$P(A=0, B=1) = 0.3$$

$$P(A=1, B=0) = x$$

$$P(A=1, B=1) = y$$

$$x + y + 0.1 + 0.3 = 1$$

$$x + y = 0.6$$



$$x + 3x = 0.6 \quad \begin{matrix} P(B_0) = x = 0.1 \\ P(B_1) = y = 0.3 \end{matrix}$$

$$x = \frac{0.6}{4} = 0.15 \Rightarrow 3x = y$$

$$y = 0.45$$

$$P(B=0) = x + 0.1 = 0.25$$

$$P(B=1) = y + 0.3 = 0.75$$

$$P(A=1) = \sum_b P(A=1, B=b) = x + y = 0.6$$

$$P(A=0) = \sum_b P(A=0, B=b) = 0.4$$

$$\begin{cases} P(A_1) P(B_0) = x \\ P(A_1) P(B_1) = y \\ P(A_0) P(B_0) = 0.1 \\ P(A_0) P(B_1) = 0.3 \end{cases}$$

e.g.

$$\begin{aligned}
 P(\bar{T}) &= 0.6 \\
 P(S|T) &= 0.3 \\
 P(S|\bar{T}) &= 0.6 \\
 P(T|S) &= \frac{1}{4}
 \end{aligned}$$

$$\begin{aligned}
 P(T|S) &= \frac{P(S|T) P(T)}{P(S)} \\
 P(S) &= P(S, T) + P(S, \bar{T}) \\
 &= P(S|T) P(T) + P(S|\bar{T}) P(\bar{T})
 \end{aligned}$$

e.g.

$$\begin{aligned}
 P(u, v, w, x, y) \\
 &= P(u) P(v|u) P(w|u, v) P(x|u, v, w) \\
 &\quad P(y|u, v, w, x)
 \end{aligned}$$

e.g. 3 tosses ind.

T: 2 or more heads

S: 2 or more tails

$$P(T, S) = 0$$

e.g. joint dist.

$$f(x, y) = \begin{cases} \frac{x(1+3y^2)}{4}, & 0 \leq x \leq 2, 0 \leq y \leq 1 \\ 0, & \text{otherwise} \end{cases}$$

$$\begin{aligned}
 P(x) &= \int_0^1 f(x, y) dy = \frac{x}{2} \\
 P(y) &= \int_0^2 f(x, y) dx = \frac{1+3y^2}{2} \\
 E[x] &= \int_0^2 x P(x) dx = \frac{4}{3} \\
 E[y] &= \int_0^1 y P(y) dy = \frac{5}{8} \\
 E[xy] &= \int_0^2 \int_0^1 xy f(x, y) dy dx = \frac{5}{6} \\
 \text{Covar}(x, y) &= E[xy] - E[x]E[y] = 0
 \end{aligned}$$

Hence, x, y are uncorrelated.

Bayes' Net : Directed Graphical Model (acyclic)
(DAG)

Conditional Distribution for each node.

$$P(x) = \prod_{i=1}^{|x|} P(x_i | \text{parents}(x_i))$$

3 types of reasoning:
 { Causal Reasoning
 { Evidential Reasoning
 { Explaining Away

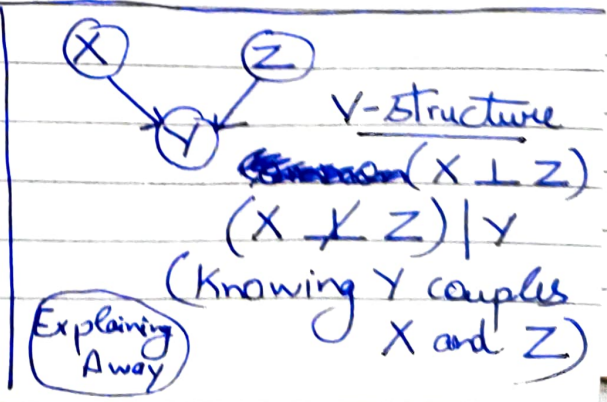
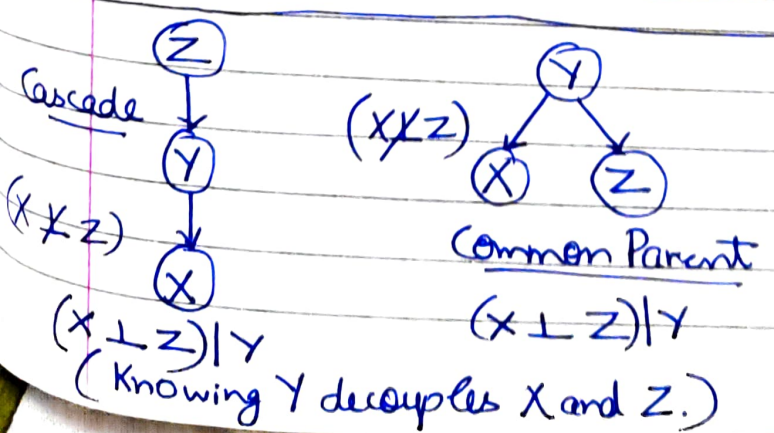
Direct connection: (They are correlated)

(A node is always dependent on its parents)

Ancestors: $E \rightarrow C \rightarrow G \rightarrow L \rightarrow S$
 $P(S | L, G, C, E) = P(S | L)$

Descendants: (the effect gives information about the cause)

Given its parents, a node is independent of all variables except its descendants.



Markov Blanket:

Co-parents: parents of a node's children.

Markov Blanket of a node is the set containing the node's parents, children and co-parents.

$$\text{Markov Blanket}(x) = \text{pa}(x) \cup \text{ch}(x) \cup \left(\bigcup_{y \in \text{ch}(x)} \text{pa}(y) \setminus \{x\} \right)$$

Theorem: A node is conditionally independent of every other node in the graph given its Markov Blanket.

D-separation:

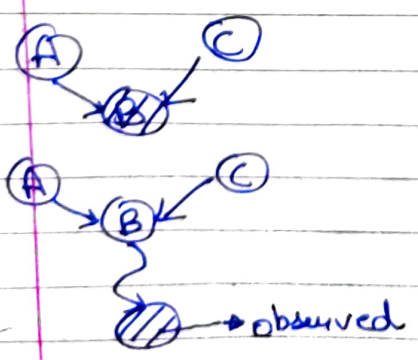
(no active paths $\equiv$ independence)

Algorithm to calculate whether $X \perp Z | Y$ by looking at graph separation.

A path is active if each Triple is active.

Triples:

- Active**
- Causal chain ($A \rightarrow B \rightarrow C$), unobserved B.
 - Common cause ($A \leftarrow B \rightarrow C$), unobserved B.
 - Common effect ($A \rightarrow B \leftarrow C$), B or one of its descendants is observed.

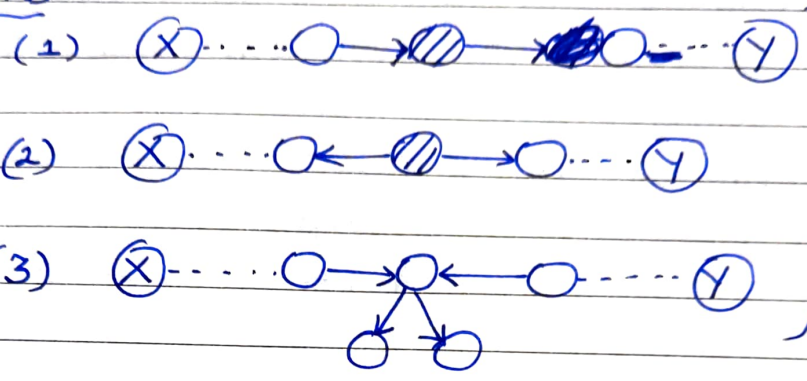


A single inactive segment can block a path.

Theorem: If there is no active path between X and Z then X and Z are d-separated given Y.

If X and Y are d-separated by Z , then X and Y are conditionally independent given Z .

Blocked paths:



shaded: ~~observed~~
 observed

inactive

A.I.-Search Algorithms :

- A search problem consists of:

- 1) State Space.
- 2) A successor function.
- 3) Start state and a goal test.

- A solution is a sequence of actions which transforms the start state to a goal state.

- Fringe: a data structure used to store all the possible states (nodes) that you can go from the current state.

- Uninformed Search:
 - Breadth First
 - Uniform Cost
 - Depth First
 - Depth Limited
 - Iterative Deepening

- Properties of Search:

- 1) Completeness. (find a soln if one exists)
- 2) Optimality. (least cost soln)
- 3) Time Complexity.
- 4) Space Complexity.

- A state space graph represents all unique states, while a search tree may repeat states.

- branching factor = no. of child nodes generated from a single state.

- Informed Search $\equiv$ Heuristic Search.

e.g. If a search tree has depth d and a branching factor b , what is the max. no. of nodes in the tree?

$$1 + b + b^2 + \dots + b^d \\ = \frac{b^{d+1} - 1}{b - 1}$$

e.g. If a search tree has a branching factor of 3 and a depth of 4, how many nodes are there at the fourth level?
Ans: 3^4

- Depth-First Search: expand deepest node first. fringe is a LIFO Stack.

b = branching factor

m = maximum depth.

$$\text{no. of nodes in entire tree} = 1 + b + b^2 + \dots + b^m \\ = O(b^m)$$

$$T.C = O(b^m) \quad (\text{All nodes in worst case})$$

$$\text{Space Complexity} = \text{max. length of fringe} \\ = O(b * m)$$

If there is a cycle in the graph then search tree could be infinite. (even if graph is finite).

• Complete only if depth is finite.

• Not optimal.

- Depth Limited Search: (DFS with Depth bound).

If solution is beneath the depth bound, the search cannot find the goal (Incomplete).
Not Optimal.

$$T.C. = O(b^L), \quad S.C. = O(b * L); \quad L \text{ is the cutoff.}$$

- Breadth First Search: (expand a shallowest node first)
(Fringe is a FIFO Queue).

s = depth of shallowest goal node.

Time Complexity = $O(b^s)$ = Space Complexity
(the last tier)

It is complete.
It is optimal if all costs are 1.

- Uniform Cost Search: expand a cheapest node first.
(Same as Dijkstra's Algo).

Fringe is a Priority Queue.

Terminate only once the goal is popped from the priority queue.

Process all nodes with cost less than cheapest solution.

If that solution costs C^* and arcs cost at least ϵ , then the "effective depth" is roughly C^*/ϵ .

Time Complexity = $O(b^{C^*/\epsilon})$ = Space Complexity.

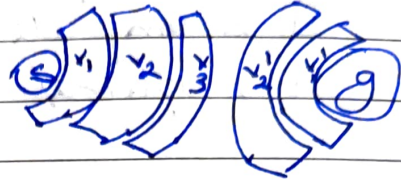
UCS is complete and optimal.

- Graph Search v/s Tree Search:

- The problem is always a graph.
- In graph search you hold a list of explored nodes, while in Tree Search, you don't.
- Tree search can visit a state multiple times.

e.g. Graph search DFS is complete while Tree search DFS is not.

- Bidirectional Search: BFS from both start and goal, simultaneously. We have a check to see whether the frontiers of the two searches intersect.



- Works well only when there are unique start and goal states.
- Requires ability to generate "predecessor" states.
- d = depth of shallowest goal node.
- Both backward and forward search will meet after ~~each~~ each search expands $d/2$ levels.

Space and Time complexity = $O(b^{d/2})$
 Complete if BFS is used in both searches.
 Optimal if BFS and paths have uniform cost (1 cost each).

- Iterative Deepening Search:

(multiple iterations)

Starting at depth limit $L=0$, iteratively increase depth limit, perform DFS with ~~that~~ each depth bound.

Memory usage same as DFS.

Space complexity = $O(b \cdot d)$

Time complexity = $1 + (1+b) + (1+b+b^2) + \dots + (1+b+\dots+b^d)$
 (a little worse than BFS) = $O(b^d)$

Complete.

Optimal if all operators have the same cost.

The states at level d have no successors generated, the states at level $d-1$ have successors generated just once, and so on. The states at level $d-k$ have their successors generated k times.

- Informed Search Techniques:
 - Greedy Best First
 - A^*
 - Iterative Deepening A^*

Heuristic:

A search heuristic $h(n)$ is an estimate of the cost of the optimal (cheapest) path from node n to a goal node.

- Best first Search: Use an evaluation function $f(n)$ for node n .

Always choose the node from fringe that has the lowest f value.

In BFS: $f(n) = \text{depth}(n)$

UCS: $f(n) = \text{sum of edge costs from start to } n = g(n)$

- Greedy best first search: $O(b^m)$ [depends on $h(n)$]
(always select the node with smallest $h(n)$ value).

* not necessarily optimal

(not complete)

i.e., $f(n) = h(n)$

[heuristic function]

S.C. = keeps all nodes in memory = $O(b^m)$

$g(n)$ = Total cost from starting to node n
 $h(n)$ = Estimated cost from current node n to the goal node.
 $f(n)$ = Evaluation function.

- Beam Search: $f(n) = h(n)$
 maximum size of the nodes list is k
 Only keeps k best nodes as candidates for expansion, and throws the rest away.
 More space efficient than greedy search, but may throw away a node that is on a solution path.

k = beam width

$k=1$ (Hill Climbing Search)

$k=\infty$ (Best First Search)

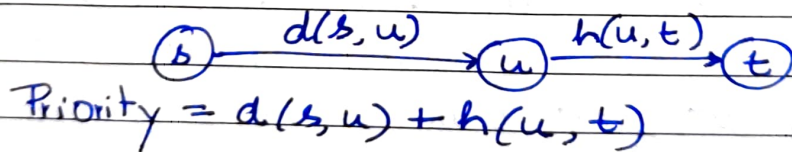
Not Complete.

Not Optimal.

Time Complexity = $O(k \cdot m)$
 (m = max. depth of any path in search tree).

Space Complexity = $O(k \cdot m)$

- A* Algorithm:



Evaluation fn for A*

$$f(n) = g(n) + h(n)$$

Common heuristics in A*: Manhattan Distance, Euclidean Distance.

- When should A* terminate?
 Only stop when we dequeue a goal.

Admissible Heuristic function:

$h^*(n)$ = cost of the optimal path from node n to any goal node.

$$\boxed{h(n) \leq h^*(n)} \quad \forall n$$

$h(n) \leq \text{true cost}$

(our $h(n)$ is underestimate)

Consistent Heuristic function:

distance b/w any pair of nodes is greater than or equal to the difference in their heuristics.

$\forall p, q$,

$$\boxed{|d(q) - d(p)| \geq |h(q) - h(p)|}$$

- Both A^* tree search and graph search are complete.
- Inadmissible heuristic does not guarantee optimality.
- If $h(n)$ is admissible:
 - A^* using tree search is optimal
 - A^* using graph search may not be optimal.
- Consistency implies admissibility.
Admissibility does not imply consistency.
- If $h(n)$ is consistent:
 - Both A^* tree and graph search are optimal.

for optimality of A^* :

Tree search: Need admissability.
Graph search: Need consistency.

e.g. If h_1, h_2 are admissible,
 $\max(h_1, h_2)$
 $\min(h_1, h_2)$
 $|h_1 - h_2|$
are also admissible.

e.g. Using heuristic h on a graph $G(V, E)$ with non-negative edge weights, the A^* algorithm succeeds in finding the shortest path from any vertex s to any other vertex t .

Suppose h is replaced with h' . Select all for which h' is guaranteed to still find the shortest path.

- a) $h' = 0$
- b) $h' = -1$
- c) $h' = h/2$
- d) $h' = \sqrt{h}$

[A, B, C is correct answers.]

- A^* is a modification of Dijkstra's Algorithm that is optimized for a single destination.

- The smaller $h(n)$ is, the more nodes A^* will expand. (If h is admissible).

- Iterative Deepening A*: (Runs DFS)
Depth bound is measured in terms of the f value.

Increase the depth bound: to the minimum of the f -values that exceeded the previous bound.
Never expand any node n with $f(n) > F$ (limit)

Complete

Find optimal is admissible (for tree search).

More costly than A* in general.

(But requires less memory than A*)

e.g. If optimal goal is reached after d actions.
How many times will the initial state be expanded (in worst case)

ID DFS $\rightarrow d$

IDA* $\rightarrow b^d$

• Adversarial Search:

Games as Search Problems:

- States
- Initial state
- Successor function
- Terminal Test
- Utility function
- Solution

One player maximizes result.

The other player minimizes result.

Minimax Search:

Players alternate.

The computer is MAX.
The opponent is MIN.

- move : a move by both players
- ply : a half move
- utility func : the func. applied to leaf nodes
- backed-up value:
 - of a max-pos : value of its largest successor.
 - of a min-pos : value of its smallest successor.

- Minimax Value(n) =
$$\begin{cases} \text{Utility}(n) & \text{if } n \text{ is a terminal state} \\ \text{Max}_{s \in \text{Succ}(n)} \text{Minimax Value}(s) & \text{if } n \text{ is MAX node} \\ \text{Min}_{s \in \text{Succ}(n)} \text{Minimax Value}(s) & \text{if } n \text{ is MIN node} \end{cases}$$

We can use DFS.

- Properties of MiniMax :
 - Complete $\rightarrow$ Yes (if tree is finite)
 - Optimal $\rightarrow$ Yes (against an optimal opponent)
No (does not exploit opponent weakness against suboptimal opponent).
 - Time Complexity $\rightarrow O(b^m)$
 - Space Complexity $\rightarrow O(b^*m)$

- Alpha-Beta pruning applied to a minimax tree:
 - Returns the same decision as minimax.
 - Prunes branches that cannot influence final decision.

Alpha: Value of best (highest value) choice we have found so far on the path for MAX.

Beta: Value of best (lowest value) choice we have found so far on path for MIN.

- Initially, $[\alpha = -\infty, \beta = \infty]$ (we terminate/prune when (if) $\alpha \geq \beta$)

The order in which moves are evaluated is important.

Bad Move ordering might prune nothing.

With perfect node ordering we can reduce time complexity to $O(b^{m/2})$.

Evaluation function:

e.g. for chess, linear weighted sum of features.

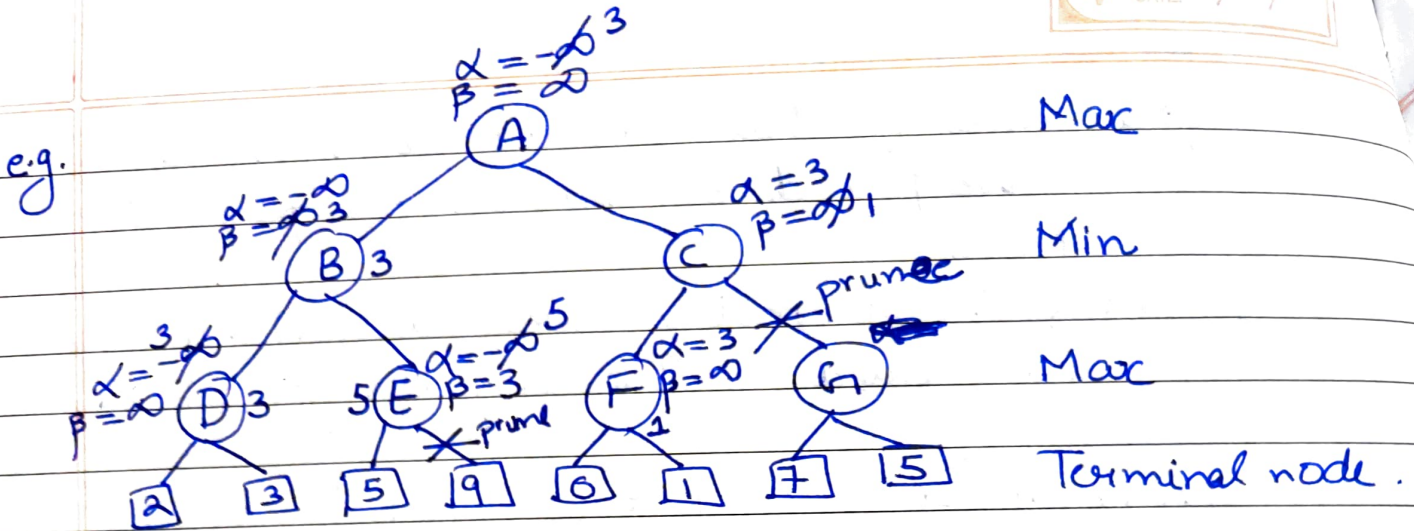
$$\text{Eval}(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

- Heuristic evaluation function + cutoff test

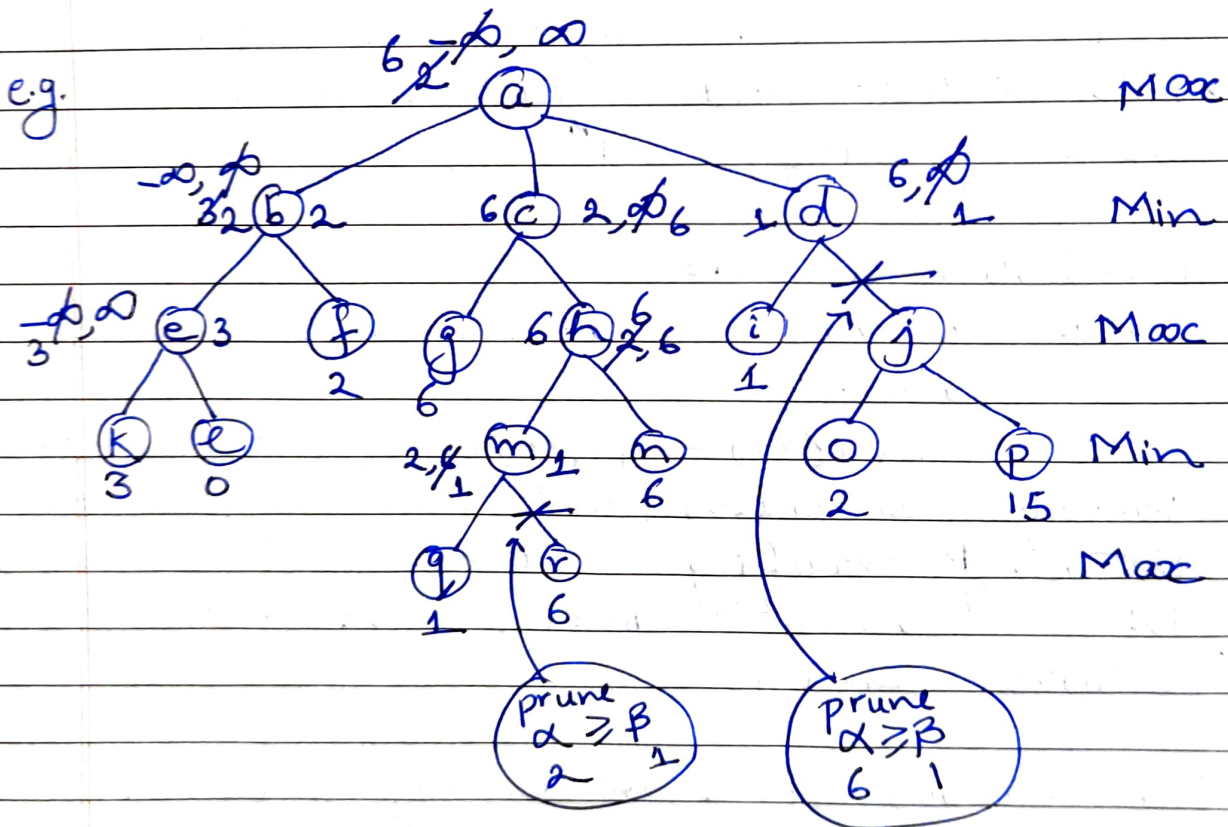
Instead of searching until we find a terminal state, we can cut search sooner and apply the evaluation function.

- For stochastic games we use Expectiminimax.
(at chance nodes we compute the expected value).

$$E(x) = \sum_{x \in X} P(x) x$$



When $[\alpha \geq \beta] \Rightarrow$ We prune



We prune subtrees rooted at r and j.

A*: $f(n) = g(n) + \alpha h(n)$ (h/n is given as admissible)
Tree Search:

The path length is
at most α times
the optimal cost.

Modular Arithmetic:

Let a be an integer and d a positive integer. Then there are unique integers q and r , with $0 \leq r < d$, such that $a = dq + r$

$$\boxed{r = a \bmod d}$$

If two numbers a and b leave the same remainder when divided by a third number m , then we say " a is congruent to b modulo m ", and write

$$a \equiv b \pmod{m}$$

$$\begin{aligned} \text{if } a \bmod n &= r \\ \text{then } (a + kn) \bmod n &= r \end{aligned}$$

$$\begin{aligned} \text{if } a &\equiv b \pmod{n} \\ a + k_1 n &\equiv b + k_2 n \pmod{n} \end{aligned}$$

$$\text{e.g. } a \equiv b \pmod{n}$$



$$\begin{aligned} a &= nk_1 + r \\ b &= nk_2 + r \end{aligned} \Rightarrow \begin{aligned} a - b &= n(k_1 - k_2) \\ &= nk \end{aligned}$$

$$\Rightarrow (a - b) \bmod n = 0$$

$$\text{e.g. } a \equiv b \pmod{n}$$

$$\Rightarrow \underbrace{a \bmod n}_r \equiv \underbrace{b \bmod n}_r \pmod{n}$$

$$\Rightarrow ((a \bmod n) \bmod n) \bmod n \dots \equiv b \bmod n \pmod{n}$$

Theorem:

If $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$, then $a+c \equiv b+d \pmod{m}$.

If $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$, then $axc \equiv bxd \pmod{m}$.

e.g. True / False

- ① $a+n \equiv b \pmod{n}$ ✓
- ② $a+3n \equiv b+2n \pmod{n}$ ✓
- ③ $a+k \equiv b+n \pmod{n}$ X
- ④ $a+k+3n \equiv b+k \pmod{n}$ ✓
- ⑤ $a+2k \equiv b+k \pmod{n}$ X

e.g. $a \pmod{m}$

$\Rightarrow a \pmod{m} \cdot c \pmod{m} \pmod{m}$

e.g. If we know that $a \equiv 3 \pmod{7}$ and $b \equiv 4 \pmod{7}$, then we can determine that $ab \equiv 12 \equiv 5 \pmod{7}$

e.g. $a \equiv b \pmod{n}$

$k \equiv k \pmod{n}$

$\Rightarrow ak \equiv bk \pmod{n}$

can't go this way

e.g. $3x+4 \equiv 2x+8 \pmod{9}$

$\Rightarrow x+4 \equiv 8 \pmod{9}$

$\Rightarrow x \equiv 4 \pmod{9}$

$x = 9k+4$

$$\begin{aligned} \text{e.g. } (100)^5 \bmod 7 &= (100 \bmod 7)^5 \bmod 7 \\ &= 2^5 \bmod 7 \\ &= 32 \bmod 7 \\ &= 4 \end{aligned}$$

$$\text{e.g. } 994 \cdot 996 \cdot 997 \cdot 998 \equiv (-6)(-4)(-3)(-2) \equiv 144 \pmod{1000}$$

$$\begin{aligned} \text{e.g. } 17^{753} \bmod 9 &= (-1)^{753} \bmod 9 \\ &= -1 \bmod 9 = 8 \end{aligned}$$

$$\begin{aligned} 17^{753} \bmod 9 &= 8^{753} \bmod 9 \\ &= 8 \cdot 64^{376} \bmod 9 \\ &= 8 \cdot 1^{376} \bmod 9 = 8 \end{aligned}$$

$$\begin{aligned} \text{e.g. } 13^{99} \bmod 17 &= (-4)^{99} \bmod 17 \\ &= (-4) 16^{49} \bmod 17 \\ &= (-4) (-1)^{49} \bmod 17 \\ &= 4 \end{aligned}$$

$$\begin{aligned} \text{e.g. } 3^{51} \bmod 5 &= (-2)^{51} \bmod 5 \\ &= (-2) 4^{25} \bmod 5 \\ &= (-2) (-1)^{25} \bmod 5 \\ &= 2 \end{aligned}$$

$$\begin{aligned} \text{e.g. } 7^{100} \bmod 10 &= (-3)^{100} \bmod 10 \\ &= 9^{50} \bmod 10 \\ &= (-1)^{50} \bmod 10 = 1 \end{aligned}$$

$$\begin{aligned} \text{e.g. } 2017^{2017} \bmod 10 &= 7^{2017} \bmod 10 \\ &= (-3)^{2017} \bmod 10 \\ &= (-3) 9^{1008} \bmod 10 \\ &= -3 \bmod 10 = 7 \end{aligned}$$

Inference:

• Exact inference by enumeration:

probabilistic inference:

"compute a desired probability from other known probabilities (e.g. conditional from joint)"

e.g.

Posterior prob.

$$P(Q \mid E_1=e_1, \dots, E_k=e_k)$$

$\uparrow$ query $\uparrow$ evidences

- Enumeration (exact, exponential complexity)
- Variable elimination (exact, worst case: exp.T.C; often better)
- Sampling (approximate)

• Evidence Variables: $E_1 \dots E_k = e_1 \dots e_k$

Query Variables: Q

Hidden Variables: $H_1 \dots H_r$

We want; $P(Q \mid e_1 \dots e_k)$

$$P(Q \mid e_1 \dots e_k) = \sum_{h_1 \dots h_r} P(Q, h_1 \dots h_r, e_1 \dots e_k)$$

$$\alpha = \frac{1}{Z}$$

(normalization)
(constant)

We multiply $\frac{1}{Z}$ for normalization.

• The no. of possible assignments is exponential in the unobserved variables.

n unobserved variables $\rightarrow d^n$
 d values

Variable Elimination:

Move summations inwards as far as possible.

e.g. $P(B | j, m) = \alpha \sum_{e, a} P(B) P(e) P(a | B, e) P(j | a) P(m | a)$

Do the calculation from the inside out.

- Track objects called factors.

Initial factors are local conditional probability tables (one per node).

Any known values are selected.

Procedure: Join all factors, eliminate all hidden variables, normalize.

e.g. Join on R.

$$P(R) \times P(T|R) \Rightarrow P(R, T)$$

Eliminate R (sum over R)

$$\sum_R P(R, T) \implies P(T)$$

$$\sum_t P(L|t) \sum_r P(r) P(t|r)$$

join our

Eliminate r

join on t

Eliminate t

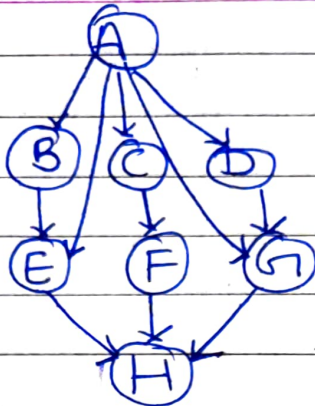
Variable elimination

Ordering of elimination of hidden variables can affect size of factors generated.

Worst case: exponential in the size of the Bayes' net.

Worst case: exponential in the size of the Bayes' net.

e.g.



Carry out ~~the~~
variable elimination
inference for $P(H|+f)$.

$$P(A) P(B|A) P(C|A) P(D|A) P(E|A,B) P(+f|C) \\ P(G|A,D), P(H|E,+f,G)$$

eliminate A to make factor f_1

$$f_1(B,C,D,E,G) = \sum_a P(a) P(B|a) P(C|a) P(D|a) \\ P(E|a,B) P(G|a,D)$$

eliminate B to make a new factor f_2

$$f_2(C,D,E,G) = \sum_b f_1(b,C,D,E,G)$$

eliminate C to make f_3

$$f_3(D,E,+f,G) = \sum_c f_2(C,D,E,G) P(+f|c)$$

eliminate D to make f_4

$$f_4(+f,E,G) = \sum_d f_3(d,E,G,+f)$$

eliminate E to make f_5

$$f_5(+f,G,H) = \sum_e f_4(e,G,+f) P(H|e,+f,G)$$

eliminate G to make f_6

$$f_6(+f,H) = \sum_g f_5(+f,g,H)$$

Now;

$$P(H|+f) = \alpha f_6(+f,H)$$

→ normalization constant.

f_1 has largest no. of unobserved variables.
no. of table entries required for $f_1 = 2^5$.

From the Bayesnet we get. (I under line whenever I eliminate that term)

$$P(A, \dots, H) = P(A) P(B|A) P(C|A) P(D|A) P(e|B, C) P(F|C) P(G|e, F) P(h|G)$$

Query: $P(B, D | E=e, H=h)$

We need to eliminate all hidden variables using variable elimination.

(Assume alphabetical ordering)

Hidden are:
A, C, F, G

$$f_1 = \sum_A P(A) P(B|A) P(C|A) P(D|A) = f_1(B, C, D)$$

$$f_2 = \sum_C f_1(B, C, D) P(e|B, C) P(F|C) = f_2(B, D, e, F)$$

$$f_3 = \sum_F f_2(B, D, e, F) P(G|e, F) = f_3(B, D, e, G)$$

$$f_4 = \sum_G f_3(B, D, e, G) P(h|G) = f_4(B, D, e, h)$$

Now, after eliminating hidden vars, the following factors remain.

(We write the joint for query & evidence vars using the remaining (not-eliminated) stuff.)

$$P(B, D, e, h) = f_4(B, D, e, h)$$

We can obtain conditional by normalizing.

$$P(B, D | e, h) = \frac{1}{Z} f_4(B, D, e, h)$$

Z ← (normalization constant)

Suntext
 f_1 : 3 vars $\rightarrow 2^3$ rows
 f_2 : 4 vars but one is observed $\rightarrow 2^3$ rows
 f_3 : 2^3 f_4 : 2^2

Teacher's Signature

constant)

Largest factors are f_1, f_2, f_3
 (2^3) (size 3)

Approximate Inference: (Sampling) (need only polynomial time complexity)

Sampling:

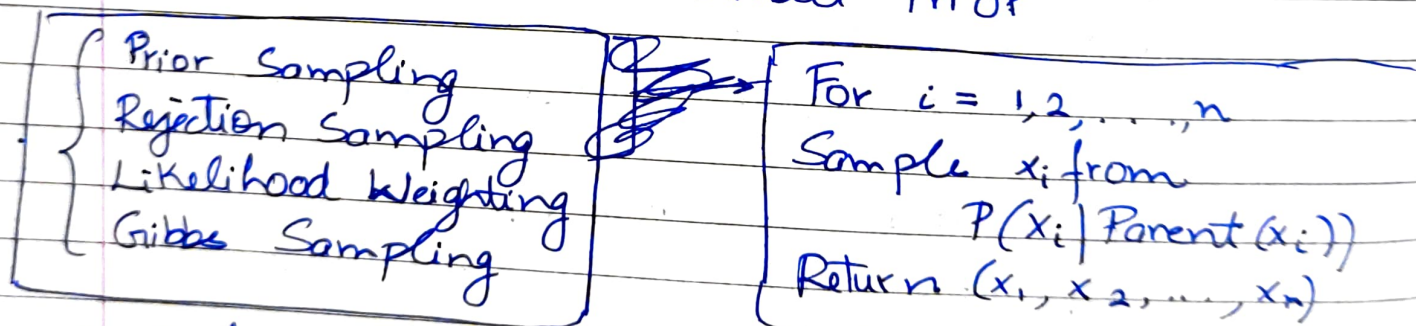
(random number generation)

- Draw N samples from a known distribution P .
- Generate N random numbers from a known distribution S .

Estimation:

- Compute an approximate probability $\hat{P}$, which approximates the real posterior probability $P(X|E)$.

$$\underbrace{P(\theta|x)}_{\text{Posterior}} \propto \underbrace{P(x|\theta)}_{\text{Likelihood}} \cdot \underbrace{P(\theta)}_{\text{Prior}}$$



e.g. To find prior prob. (from samples)

$$P(+a, -b, +c) = \frac{N(+a, -b, +c)}{N}$$

Rejection Sampling is used to estimate conditional prob.

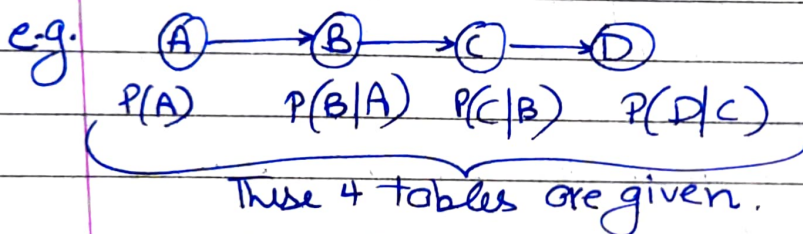
e.g. $P(c | +s)$ { ignore (reject) samples which don't have $S = +s$. }

Likelihood Weighting:

• Idea: fix evidence variables and sample the rest.

Problem: sample distribution non consistent.

Solution: weight by probability of evidence given parents.



Using Likelihood Sampling to estimate $P(-a|+b, -d)$, the following samples we obtained. Find their weights.
 $(+b, -d \text{ are evidence})$

$\langle -a, +b, +c, -d \rangle$: $\text{Weight} = \underbrace{1 * P(B|A) * 1 * P(D|C)}_{\substack{\text{prob. of} \\ \text{(evidence/parents)} \\ \text{otherwise '1'}}$

if a is also evidence take $P(A)$.

$$w_1 = P(+b|-a) \cdot P(-d|+c)$$

$$\langle +a, +b, +c, -d \rangle: w_2 = P(+b|+a) \cdot P(-d|+c)$$

$$\langle +a, +b, -c, -d \rangle: w_3 = P(+b|+a) \cdot P(-d|-c)$$

$$\langle -a, +b, -c, -d \rangle: w_4 = P(+b|-a) \cdot P(-d|-c)$$

To estimate: $P(-a|+b, -d) = \frac{w_1 + w_4}{w_1 + w_2 + w_3 + w_4}$

• In likelihood weighting, evidence influences the choice of downstream variables, but not upstream ones.

• Gibbs Sampling:

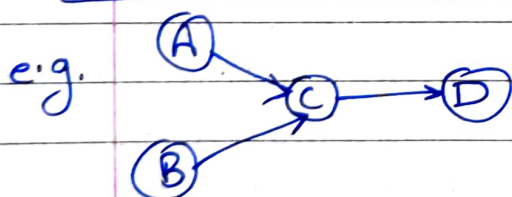
Keep track of a full instantiation $x_1, x_2, \dots, x_n$. Start with an arbitrary instantiation consistent with the evidence. Sample one variable at a time, conditioned on all the rest, but keep evidence fixed. Keep repeating this for a long time.

In the limit of repeating this infinitely many times, the resulting sample is coming from the correct distribution.

Both upstream and downstream variables condition on evidence.

In contrast likelihood weighting only conditions on upstream evidence.

• Gibbs sampling updates one variable at a time and never changes the evidence.



We have, $A = +a, D = +d$
observed variables
(evidence)

Samples: $\langle +a, -b, +c, +d \rangle: w_1 = P(+a) \cdot P(+d | +c)$

$\langle +a, -b, -c, +d \rangle: w_2 = P(+a) \cdot P(+d | -c)$

$\langle +a, +b, -c, +d \rangle: w_3 = P(+a) \cdot P(+d | -c)$

$$P(-b | +a, +d) = \frac{w_1 + w_2}{w_1 + w_2 + w_3}$$

$P(C | +a)$: Use Likelihood Weighting.

$P(C | +d)$: Use Gibbs Sampling.

$$\text{LCM}(a, b) = \frac{a \times b}{\text{HCF}(a, b)}$$

The 3 hands of clock coincide approx every 65 minutes

Aptitude:

Simple interest

$$(SI = \frac{PRT}{100})$$

Compound interest

$$Q: \frac{PR(3)}{100} = 8000 - P$$

$$\frac{3}{4} = \frac{8000 - P}{8500 - P}$$

$$\frac{PR(4)}{100} = 8500 - P$$

$$\begin{aligned} 8500(3) - 3P \\ = 8(4)1000 \\ - 4P \end{aligned}$$

$$Q: \begin{array}{lll} 9\% & 3y & SI_1 \\ 12\% & 5y & SI_2 \end{array}$$

$$\begin{aligned} P &= 32000 \\ &- 25500 \\ &= 6500 \end{aligned}$$

$$SI_2 - SI_1 = 660$$

$$P \left[\frac{(12)5}{100} - \frac{9(3)}{100} \right] = 660$$

$$P = \frac{660 \times 100}{33} = 2000$$

Rule of 72:

$$\text{year} = \frac{72}{\text{Rate}} \text{ (double)}$$

Rule of 144:

$$\text{year} = \frac{144}{\text{Rate}} \text{ (triple)}$$

C.I. compounded annually.

$$A = P \left(1 + \frac{R}{100} \right)^T$$

$$P \left(1 + \frac{R}{100} \right)^T = A$$

Percentage

Profit, Loss & Discount

$$\text{Expenditure} = \text{Price} \times \text{Consumption}$$

Ratio & Proportion

$$\star \text{ Product of 2 nos.} = \text{LCM} \times \text{HCF}$$

$$\frac{MDH}{WR} = \text{constant}$$

$$\text{Revenue} = \text{Profit} + \text{Investment}$$

$$Q: \sum_{j=0}^3 2j = \sum_{i=1}^4 2(i-1)$$

$$j=0 \rightarrow 1, 1 \rightarrow 2, 2 \rightarrow 3, 3 \rightarrow 4$$

$$\star \boxed{i=j+1} \\ 0+2+2(2)+3(2)$$

$$\sum_{i=1}^n ax_i = a \sum_{i=1}^n x_i$$

$$\sum_i (x_i + y_i) = \sum_i x_i + \sum_i y_i$$

$$\sum_{i=a}^b f(x) = \sum_{i=a}^b f(x) - \sum_{i=a}^{a-1} f(x)$$

A.P. first term, a
common diff, d

$$\boxed{a_n = a_1 + (n-1)d}$$

$$\boxed{a_n = a_{n-1} + d}$$

$$\boxed{S_n = \frac{n}{2} (2a + (n-1)d)}$$

$$\boxed{S_n = \frac{n}{2} (a_1 + a_n)}$$

$$1+2+3+\dots+n = \frac{n(n+1)}{2}$$

$$1^2+2^2+3^2+\dots+n^2 = \frac{n(n+1)(2n+1)}{6}$$

$$1^3+2^3+3^3+\dots+n^3 = \left(\frac{n(n+1)}{2} \right)^2$$

G.P. $a, ar, ar^2, ar^3, \dots$
common ratio, r

$$\boxed{a_n = ar^{n-1}}$$

$$\boxed{S_n = \frac{a(r^n - 1)}{r - 1}}$$

$$\boxed{S_\infty = \frac{a}{1-r}, (|r| < 1)}$$

$$f_z(z) = \int_{-\infty}^{\infty} f_x(x) f_y(z-x) dx$$

$$P_z(z) = \sum_x P_x(x) P_y(z-x)$$

$$Z = X + Y$$

Convolution

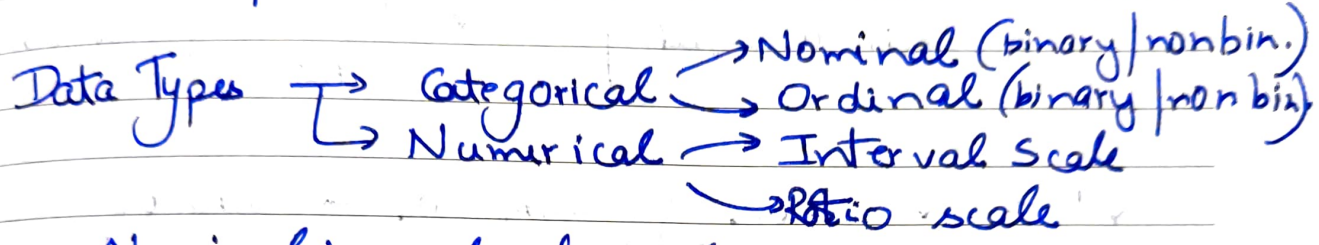
25 SUNDAY

Warehouse

OLAP:

Drill down: View more detailed data.

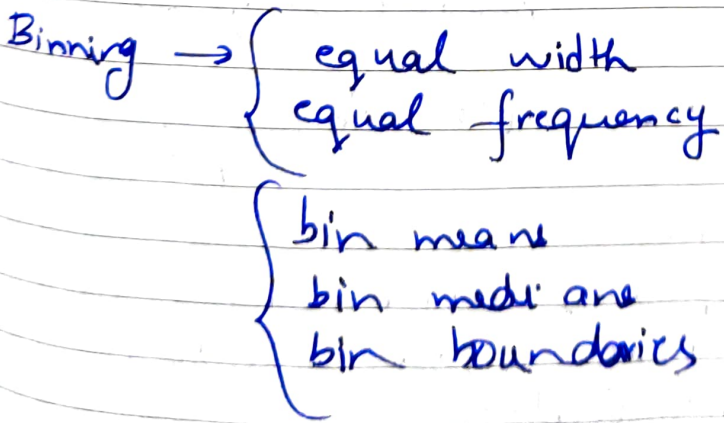
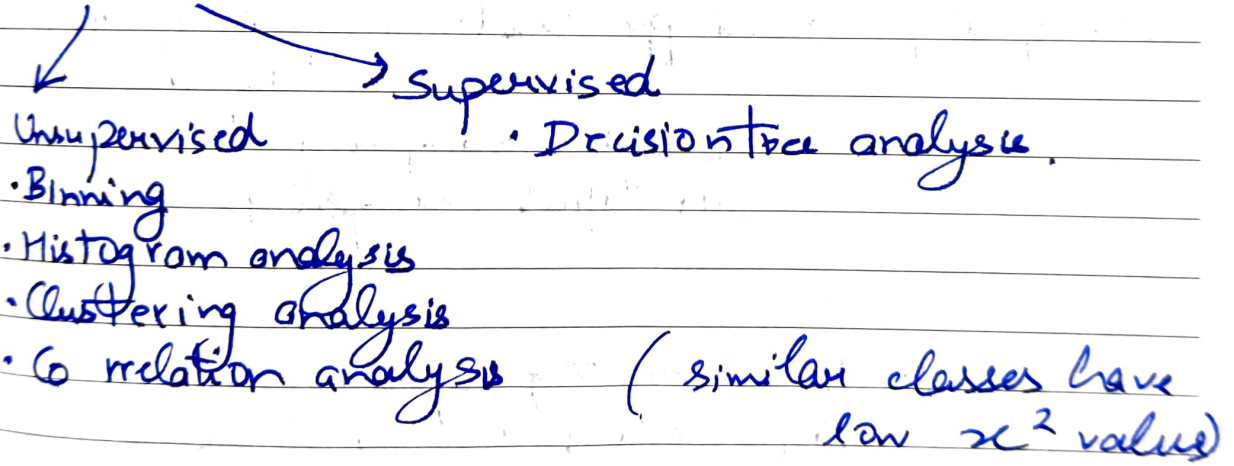
Roll Up: View more summarized data.



Nominal: male, female

Ordinal: low, mid, high.

Discretization:



$$\begin{aligned}
 \text{width} &= \frac{\text{range}}{N} \\
 &= \frac{\text{max} - \text{min}}{N}
 \end{aligned}$$

Normalization:

{ Min-Max
 z score
 Decimal Scaling $\rightarrow v' = \frac{v}{10^j}$ (all less than 1)

$$v' = \left(\frac{v - \text{min}}{\text{max} - \text{min}} \right) (\text{new_max} - \text{new_min}) + \text{new_min}$$

Sampling $\rightarrow$ Simple random (with/without replacement)
 cluster sampling
 Stratified sampling

Without ~~sampling~~ replacement:
 $Pr(\text{selecting any one of all possible samples}) = \frac{1}{N C n}$

With replacement: $Pr = \frac{1}{N^n}$

Data compression:

Lossless: Huffman, arithmetic, run length encoding
 Lossy: Quantization

M	T	W	T	F	S	S
					1	2
30	3	4	5	6	7	8
2	10	11	12	13	14	15
9	17	18	19	20	21	22
16	24	25	26	27	28	29
23						

234-132 • WK 34

WEDNESDAY

'24

AUGUST

21

Data Cube: Facts and Dimensions.

apex: most summarized

base: most specific / detailed.

Schemas: Star, Snowflake, Fact constellation

fact tables, dimension tables

contains

measurable

data and keys for

dimension tables.

has normalized dimension tables

The fact table contains foreign keys that refer to primary keys in dimension tables.

Measures: Distributive:

e.g. sum(), count(), max(), min()

Algebraic: (using distributive measures)

e.g. average()

Holistic: (cannot be computed from partitions)

(more expensive)

e.g. median(), mode(), rank(), std dev()